CERTIK

Security Assessment

Wemix governance-contract Retainer 35

CertiK Assessed on Nov 28th, 2023





CertiK Assessed on Nov 28th, 2023

Wemix governance-contract Retainer 35

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	EVM Compatible	Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Solidity	Delivered on 11/28/2023	N/A

CODEBASE

https://github.com/wemixarchive/governance-contract/

View All in Codebase Page

COMMITS

c1ac9b665fd81d5fd1edfb75daf44eb6a49df586

View All in Codebase Page

Vulnerability Summary



29	16	0	1	12	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
6	Major	6 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
6	Medium	3 Resolved, 3 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
11	Minor	7 Resolved, 1 Partially Resolved, 3 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
6	Informational	6 Resolved	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS

WEMIX GOVERNANCE-CONTRACT RETAINER 35

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Findings

CON-01 : Centralized Control of Contract Upgrade

CON-02 : Centralization Related Risks

CON-03 : Part of Staked Balance Could Be Sent to Owner Designated Address

SIU-01 : The `minimum_staking` check is incorrect

SIU-02 : Privileged Function Allows Owner to Drain All Staked Funds

SIU-03 : Initial Token Distribution

GCU-01 : Lack of Storage Gap in Upgradeable Contract

GIU-01 : Inconsistent finalization conditions

GIU-13 : Lack of Access Control

SIU-04 : NCP Staking Amount is 0

SIU-05 : The `lock()` function is missing the `ncpDeposit()` function call

SIU-06 : The `unlock()` function is missing the `ncpWithdraw()` function call

CON-04 : Upgradeable Contract Extends Non-Upgradeable Contract

CON-06 : Not all parent contract initializing functions are called

GIU-03 : Integer underflow will cause `reInitV3()` to revert

GIU-04 : Inconsistent `lockAmount` check

GIU-05 : Lack of Modifier `checkTimePeriod`

GIU-06 : Voting Threshold Not Dependent on Voting Participation

GIU-07 : Some variables of MemberInfo are not checked

GIU-12 : Out of Scope Dependency

SIU-07 : Usage of `transfer`/`send` for sending Ether

SIU-08 : Third-Party Dependency Usage

SIU-09 : Missing Check `unlockAmount` is less than `_lockedBalance`

CON-05 : Missing Emit Events

GIU-08 : Redundant Code

GIU-09 : Lack Validation for Array Length

GIU-10 : Code for test net

GIU-11 : Discussion on `weight`

SIU-10 : State Variables in Upgradeable Contracts are Initialized When Declared

Appendix

Disclaimer

CODEBASE | WEMIX GOVERNANCE-CONTRACT RETAINER 35

Repository

https://github.com/wemixarchive/governance-contract/

Commit

c1ac9b665fd81d5fd1edfb75daf44eb6a49df586

AUDIT SCOPE | WEMIX GOVERNANCE-CONTRACT RETAINER 35

6 files audited ● 3 files with Acknowledged findings ● 3 files without findings

ID	Repo	File	SHA256 Checksum
GCU	wemixarchive/governance-contract	GovChecker.sol	1b9ab9a83a4732fe7c2704a62ae39fe8a9c9ef99e8571d75456d5cb76e3e8c0e
GIU	wemixarchive/governance-contract	GovImp.sol	4d283002d841ef912e15b55841f1a5b1d65ef67ddf5201d337136e591e234d28
SIU	wemixarchive/governance-contract	StakingImp.sol	9f3f03f25d01dfebfafcc18891754cc381d8c7e552d28f5ab1f585d4da336a88
GOC	wemixarchive/governance-contract	Gov.sol	c08a31655e1ff67fde4879ea5eeb28ab2f0a62c59f8c4ba6dc2e0ecf0561dcdb
AGU	wemixarchive/governance-contract	abstract/AGov.sol	d9bc7723e05d94e760f97f11c86aaa6d477bac1ae3245d3be0e0b4745a1b5735
STK	wemixarchive/governance-contract	storage/Staking.sol	cd7fe003f4c6e7d7b73734685b6dcac0332187553d6c26206076ca39e87b0c8c

APPROACH & METHODS

WEMIX GOVERNANCE-CONTRACT RETAINER 35

This report has been prepared for Wemix to discover issues and vulnerabilities in the source code of the Wemix governance-contract Retainer 35 project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | WEMIX GOVERNANCE-CONTRACT RETAINER 35



29	0	6	6	11	6
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Wemix governance-contract Retainer 35. Through this audit, we have uncovered 29 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
CON-01	**Centralized Control Of Contract Upgrade**	**Centralization**	**Major**	**Acknowledged**
CON-02	**Centralization Related Risks**	**Centralization**	**Major**	**Acknowledged**
CON-03	**Part Of Staked Balance Could Be Sent To Owner Designated Address**	**Centralization**	**Major**	**Acknowledged**
SIU-01	The `minimum_staking` Check Is Incorrect	Logical Issue	Major	Acknowledged
SIU-02	**Privileged Function Allows Owner To Drain All Staked Funds**	**Centralization**	**Major**	**Acknowledged**
SIU-03	**Initial Token Distribution**	**Centralization**	**Major**	**Acknowledged**
GCU-01	Lack Of Storage Gap In Upgradeable Contract	Logical Issue	Medium	Acknowledged
GIU-01	Inconsistent Finalization Conditions	Inconsistency	Medium	Resolved
GIU-13	Lack Of Access Control	Access Control	Medium	Resolved
SIU-04	NCP Staking Amount Is 0	Logical Issue	Medium	Resolved
SIU-05	The `lock()` Function Is Missing The `ncpDeposit()` Function Call	Logical Issue	Medium	Acknowledged

ID	Title	Category	Severity	Status
SIU-06	The `unlock()` Function Is Missing The `ncpWithdraw()` Function Call	Logical Issue	Medium	Acknowledged
CON-04	Upgradeable Contract Extends Non-Upgradeable Contract	Logical Issue	Minor	Acknowledged
CON-06	Not All Parent Contract Initializing Functions Are Called	Volatile Code	Minor	Acknowledged
GIU-03	Integer Underflow Will Cause `reInitV3()` To Revert	Logical Issue	Minor	Resolved
GIU-04	Inconsistent `lockAmount` Check	Inconsistency	Minor	Resolved
GIU-05	Lack Of Modifier `checkTimePeriod`	Volatile Code	Minor	Resolved
GIU-06	Voting Threshold Not Dependent On Voting Participation	Logical Issue	Minor	Acknowledged
GIU-07	Some Variables Of MemberInfo Are Not Checked	Volatile Code	Minor	Resolved
GIU-12	Out Of Scope Dependency	Logical Issue	Minor	Resolved
SIU-07	Usage Of `transfer` / `send` For Sending Ether	Volatile Code	Minor	Partially Resolved
SIU-08	Third-Party Dependency Usage	Volatile Code	Minor	Resolved
SIU-09	Missing Check `unlockAmount` Is Less Than `_lockedBalance`	Volatile Code	Minor	Resolved
CON-05	Missing Emit Events	Coding Style	Informational	Resolved
GIU-08	Redundant Code	Coding Style	Informational	Resolved
GIU-09	Lack Validation For Array Length	Logical Issue	Informational	Resolved

ID	Title	Category	Severity	Status
GIU-10	Code For Test Net	Logical Issue	Informational	Resolved
GIU-11	Discussion On `weight`	Logical Issue	Informational	Resolved
SIU-10	State Variables In Upgradeable Contracts Are Initialized When Declared	Logical Issue	Informational	Resolved

CON-01 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	**Major**	**GovChecker.sol: 12; GovImp.sol: 15; StakingImp.sol: 14**	**Acknowledged**

Description

`GovImp` , `StakingImp` , and `GovChecker` are upgradeable contracts, the owner can upgrade the contract without the community's commitment. If an attacker compromises the account, he can change the implementation of the contract and drain tokens from the contract.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

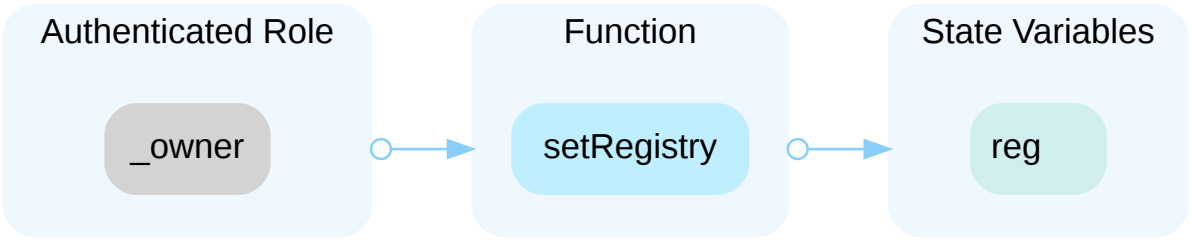
[Wemix Team]: We will use a time-lock and a multi-sig contract with Governance members.

CON-02 | CENTRALIZATION RELATED RISKS

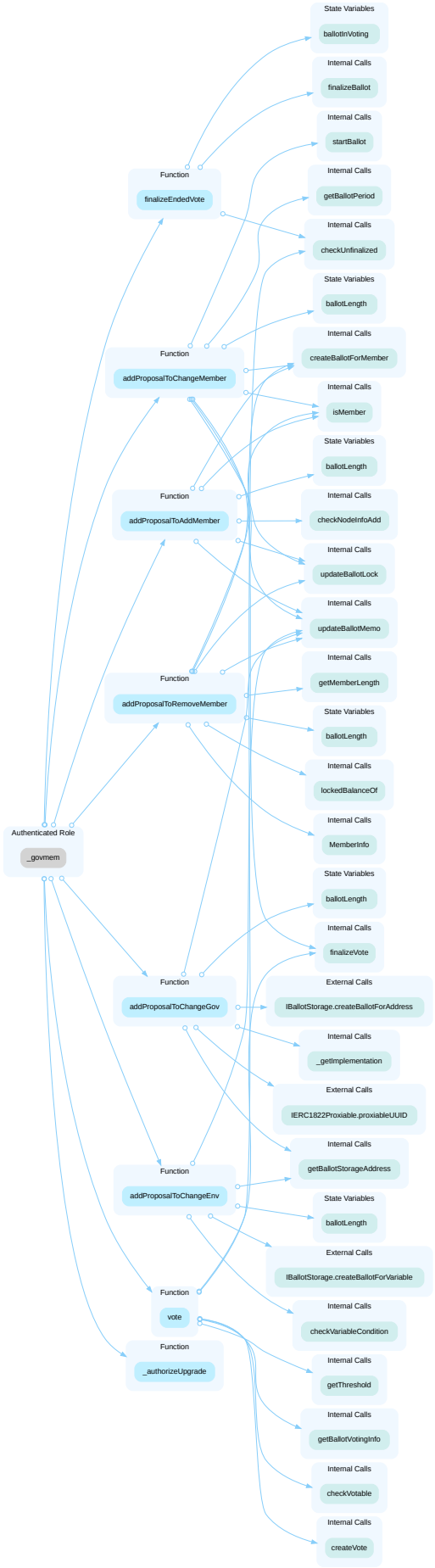
Category	Severity	Location	Status
Centralization	Major	GovChecker.sol: 31; GovImp.sol: 265, 292, 345, 397, 427, 462, 528, 1143, 1166, 1230, 1241; StakingImp.sol: 139, 241, 253, 260, 300	Acknowledged

Description

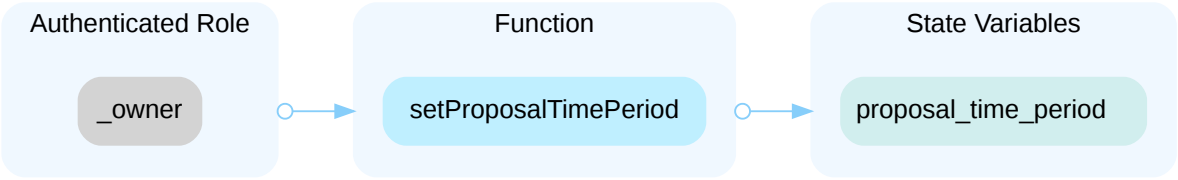
In the contract `GovChecker` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and set `Registry`.



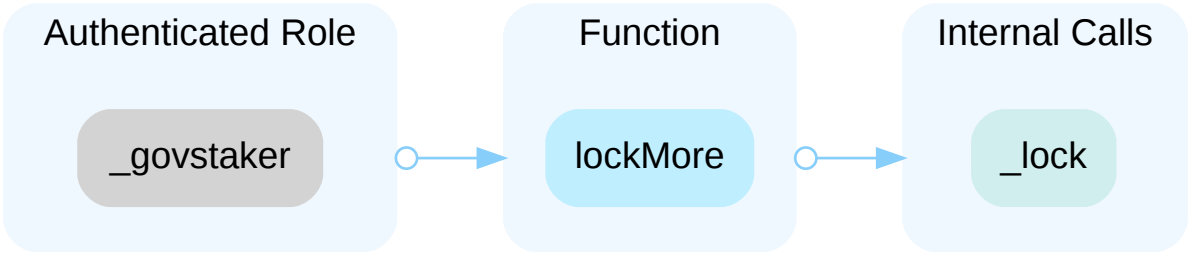
In the contract `GovImp` the role `_govmem` has authority over the functions shown in the diagram below. Any compromise to the `_govmem` account may allow the hacker to take advantage of this authority and can add proposals, vote and finalize end vote.


Authenticated Role
_govmem
Function
finalizeEndedVote
Function
addProposalToChangeMember
Function
addProposalToAddMember
Function
addProposalToRemoveMember
Function
addProposalToChangeGov
Function
addProposalToChangeEnv
Function
vote
Function
_authorizeUpgrade
State Variables
ballotInVoting
Internal Calls
finalizeBallot
Internal Calls
startBallot
Internal Calls
getBallotPeriod
Internal Calls
checkUnfinalized
State Variables
ballotLength
Internal Calls
createBallotForMember
Internal Calls
isMember
State Variables
ballotLength
Internal Calls
checkNodeInfoAdd
Internal Calls
updateBallotLock
Internal Calls
updateBallotMemo
Internal Calls
getMemberLength
State Variables
ballotLength
Internal Calls
lockedBalanceOf
Internal Calls
MemberInfo
State Variables
ballotLength
Internal Calls
finalizeVote
External Calls
IBallotStorage.createBallotForAddress
Internal Calls
_getImplementation
External Calls
IERC1822Proxiable.proxiableUUID
Internal Calls
getBallotStorageAddress
State Variables
ballotLength
External Calls
IBallotStorage.createBallotForVariable
Internal Calls
checkVariableCondition
Internal Calls
getThreshold
Internal Calls
getBallotVotingInfo
Internal Calls
checkVotable
Internal Calls
createVote

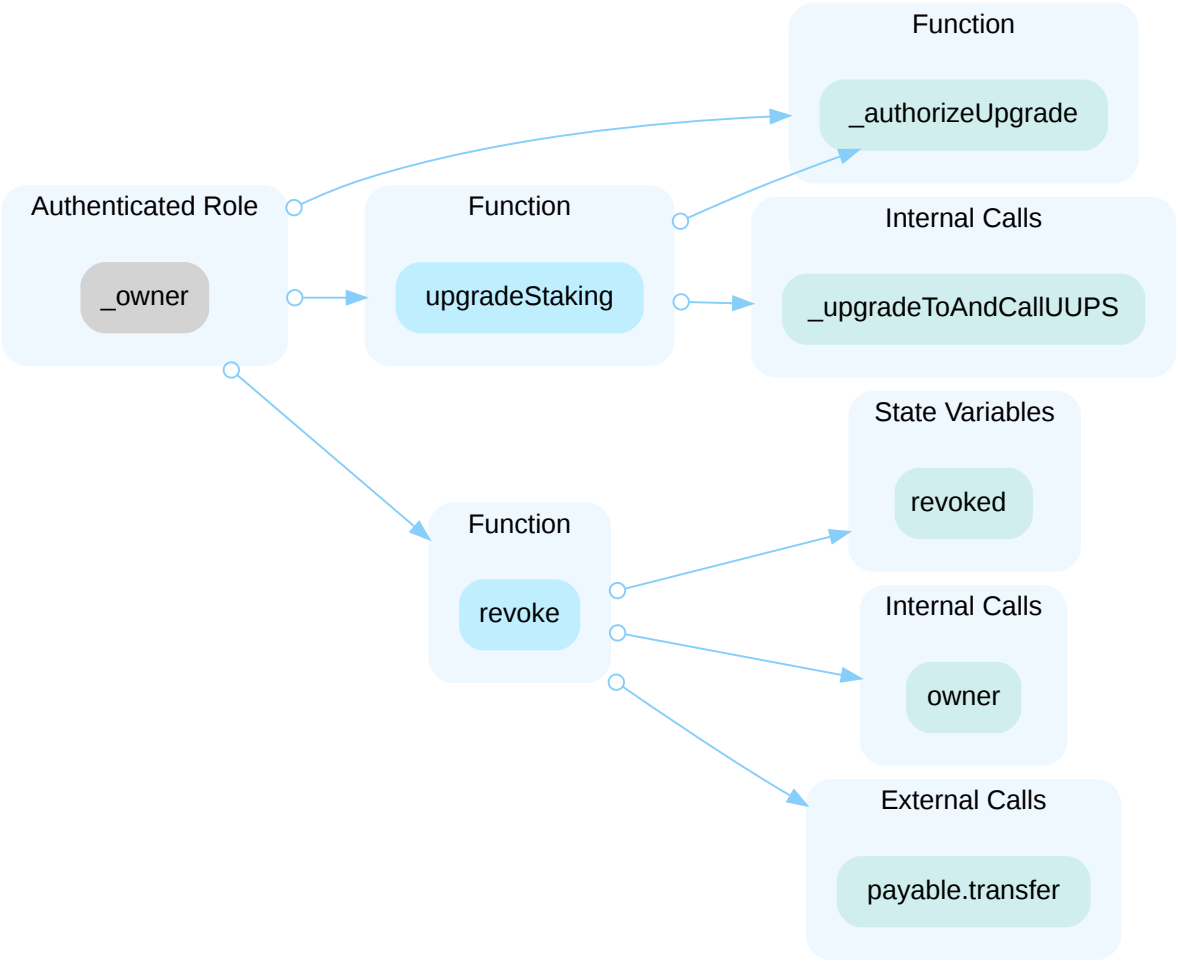
In the contract `GovImp` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and can re-init and set `posalTimePeriod`.



In the contract `StakingImp` the role `_govstaker` has authority over the functions shown in the diagram below. Any compromise to the `_govstaker` account may allow the hacker to take advantage of this authority and can lock more amounts.



In the contract `StakingImp` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and can revoke the staking contract, upgrade the staking contract, and set NCP staking address.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We recommend carefully managing the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multi-signature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term, and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key being compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness of privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement;
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles;
  OR
- Remove the risky functionality.

Alleviation

[Wemix Team]: We will use a time-lock and a multi-sig contract with Governance members.

CON-03 PART OF STAKED BALANCE COULD BE SENT TO OWNER DESIGNATED ADDRESS

Category	Severity	Location	Status
Centralization	**Major**	**GovImp.sol: 1123; StakingImp.sol: 168**	**Acknowledged**

Description

In order to withdraw the staked balance from the staking contract, the staker needs to unlock its stake which requires creating a ballot and passing the voting process. If the ballot that passes has an `unlockAmount` that is smaller than the staker's locked balance, then the difference is sent to the owner-designated reward address. Stakers might not be able to withdraw the full staked balance. Additionally, the `unlock` function is called twice on the `addr`, in line 1124 of `GovImp`, and in line 170 of `Staking`, with a total amount of `locked`, despite part of the `locked` balance being transferred to the reward pool address.

```
1122      if (locked > unlockAmount) {
1123          staking.transferLocked(addr, locked.sub(unlockAmount));
1124          unlock(addr, unlockAmount);
```

```
168     function transferLocked(address from, uint256 amount) external
override onlyGov {
169         if (amount == 0) return;
170         unlock(from, amount);
171         _balance[from] = _balance[from] - amount;
172         address ecosystem = getEcosystemAddress();
173         _balance[ecosystem] = _balance[ecosystem] + amount;
174
175         emit TransferLocked(from, amount, _balance[from], availableBalanceOf(
from));
176     }
```

Recommendation

We recommend including some restrictions on how much of the staked balance can be sent to the reward address, and a minimum percentage of staked balance that any staker can withdraw from the staking contract. We advise the client to confirm whether the total `unlock` amount is correct.

We advise the client to carefully manage the owner account and the reward address' private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol to be improved via a decentralized mechanism or via smart-contract-based accounts with enhanced security practices, e.g. Multisignature wallets. Here are some feasible solutions that would also mitigate the potential risk:

- Time-lock with reasonable latency, i.e. 48 hours, for awareness on privileged operations;
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key;
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.

Alleviation

[Wemix Team]: According to our policy, when a penalty is imposed on the member, there is a difference between the locked balance and the unlock amount. "Unlock" event occurs only when there is either "MemberChange" or "MemberRemove." When penalizing the changed/removed member, the input amount must be smaller than the staking amount. Since imposing a penalty requires a majority of votes to agree, and all governance members can check the contents of Ballot, there is little chance that an incorrectly entered vote will pass.

SIU-01 | THE `minimum_staking` CHECK IS INCORRECT

Category	Severity	Location	Status
Logical Issue	Major	StakingImp.sol: 320	Acknowledged

Description

In the `delegateDepositAndLockMore()` function, it is expected that the user's locked balance falls within the staking range. However, there is an issue in the check involving `minimum_staking` where the locked balance is not considered in addition to the user's deposit value. Consequently, if the NCP is the first to invoke the function and the locked balance is 0 while `minimum_staking` is greater than 0, the function will revert, and an error message stating "user should be in the staking range" will be displayed.

Proof of Concept

```
function testdelegateDepositAndLockMore() public {
        StakingImp stakingImp = new StakingImp();
        stakingImp.setNCPStaking(address(this));
        deal(address(this), 1);
        stakingImp.delegateDepositAndLockMore{value:1}(address(1));
    }
```

```
Test result: FAILED. 0 passed; 1 failed; finished in 11.18ms

Failing tests:
Encountered 1 failing test in test/foundry/StakingImp.t.sol:StakingImpTest
[FAIL. Reason: user should be in staking range] testdelegateDepositAndLockMore() (gas: 2374626)

Encountered a total of 1 failing tests, 0 tests succeeded
```

Recommendation

We recommend including the user's locked balance added to the deposit value when checking against `the minimum_staking` requirement, ensuring accurate validation and preventing undesired reverts.

Alleviation

[Wemix Team]: Only allow delegates if a governance member is added first and has a deposit of at least minimum_staking.

SIU-02 PRIVILEGED FUNCTION ALLOWS OWNER TO DRAIN ALL STAKED FUNDS

Category	Severity	Location	Status
Centralization	**Major**	**StakingImp.sol: 241**	**Acknowledged**

Description

The `revoke()` function is callable by the contract owner once and would send all staked balances in the staking contract to the owner's address. Any compromise to the `_owner` address would cause the loss of all stakers' funds.

Recommendation

We recommend returning the staked balance to stakers, before sending any extra funds left in the staking contract to the `_owner` address.

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Wemix Team]: We will use a time-lock and a multi-sig contract with Governance members.

SIU-03 | INITIAL TOKEN DISTRIBUTION

Category	Severity	Location	Status
Centralization	**Major**	**StakingImp.sol: 69**	**Acknowledged**

Description

When the `init()` function is invoked with a non-empty `data` variable, the `amount` of ETH tokens is sent to the specified `addr` address. This implementation carries a potential centralization risk, as the anonymous `addr` has the ability to withdraw tokens without obtaining consensus from the community. Any compromise to the `addr` account that holds undistributed tokens may allow the attacker to withdraw from the contract, resulting in other users withdrawing without sufficient amounts.

Recommendation

It's recommended the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team shall make enough efforts to restrict the access of the private key. A multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to the private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize project teams with a third-party KYC provider to create greater accountability.

Alleviation

[Wemix Team]: This function is for testing and the contract in miannet is already initialized.

GCU-01 | LACK OF STORAGE GAP IN UPGRADEABLE CONTRACT

Category	Severity	Location	Status
Logical Issue	Medium	GovChecker.sol: 12	Acknowledged

Description

There is no storage gap preserved in the logic contract. Any logic contract that acts as a base contract that needs to be inherited by other upgradeable child should have a reasonable size of storage gap preserved for the new state variable introduced by the future upgrades.

Recommendation

We recommend having a storage gap of a reasonable size preserved in the logic contract in case that new state variables are introduced in future upgrades. For more information, please refer to: https://docs.openzeppelin.com/contracts/3.x/upgradeable#storage_gaps.

Alleviation

[Wemix Team]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

GIU-01 | INCONSISTENT FINALIZATION CONDITIONS

Category	Severity	Location	Status
Inconsistency	Medium	GovImp.sol: 478~481, 628	Resolved

Description

In the `vote()` function, if the number of "accept" votes exceeds the threshold, or the number of "reject" votes exceeds the threshold, or the sum of "accept" and "reject" votes is equal to 10,000, the vote will be finalized. However, it is important to note that the `fromValidBallot()` function lacks the condition `(accept + reject) == 10000`. This condition ensures that the total number of votes adds up to 10,000, and its absence may lead to unexpected behavior or inconsistencies in the voting process.

```
462     function vote(uint256 ballotIdx, bool approval)
463         external
464         onlyGovMem
465         nonReentrant
466         checkLockedAmount
467     {
468         ...
469         if (
470             accept >= threshold ||
471             reject >= threshold ||
472             (accept + reject) == 10000
473         )
474         ...
475     }
```

```
619   function fromValidBallot(uint256 ballotIdx, uint256 targetType)
620         private
621         view
622     {
623         ...
624        require(
625             accept >= getThreshold() || reject >= getThreshold(),
626             "Not yet finalized"
627         );
628   }
```

Recommendation

We recommend add the `(accept + reject) == 10000` condition within the `fromValidBallot()` function to align with the intended logic.

Alleviation

The client revised the code and resolved this issue in commit : 9cb44a4cf4e03437afe98947f958df3e4c71ec16

GIU-13 | LACK OF ACCESS CONTROL

Category	Severity	Location	Status
Access Control	Medium	GovImp.sol: 1248, 1290, 1300	Resolved

Description

The function `setMembersForMigration()` / `setBallotForMigration()` / `setProposalTimePeriodForMigration()` can be called by anyone as it has no access restriction. This enables anyone to call these and set members, ballot, and proposal time period.

Recommendation

We recommend adding a modifier to control who can call this function or removing these functions.

Alleviation

The client revised the code and resolved this issue in commit : fdf4a7cbf8d6d4e7c34a50e27d7c894ef50cf15b

SIU-04 | NCP STAKING AMOUNT IS 0

Category	Severity	Location	Status
Logical Issue	Medium	StakingImp.sol: 91	Resolved

Description

In the `deposit()` function, if the sender is a member of the governance and the minimum lock exceeds the current locked value, the difference between the sender's locked balance and the minimum lock value will be locked. However, it is important to note that after invoking the `_lock()` function, the `_lockedBalance` is updated, resulting in the current `_lockedBalance` being equal to `minimum_staking`. This can potentially lead to a scenario where the NCP staking amount becomes 0 if `ncpStaking` is not equal to `address(0)`.

Recommendation

We recommend introducing a new variable to capture and store the value of `_lockedBalance` before executing the `_lock()` function.

Alleviation

The client revised the code and resolved this issue in commit : 0ebe4e1abd9425ca146aa41bf0f4ebaea837193a

SIU-05 THE `lock()` FUNCTION IS MISSING THE `ncpDeposit()` FUNCTION CALL

Category	Severity	Location	Status
Logical Issue	Medium	StakingImp.sol: 135	Acknowledged

Description

In the `lockMore()` function, if the sender is a staker and `ncpStaking` is not equal to `address(0)`, the `ncpDeposit()` function is invoked. However, in the `lock()` function, there is no corresponding logic to handle the situation when the payee is a staker and there is no restriction on checking whether the payee is a staker.

```
135
   function lock(address payee, uint256 lockAmount) external override onlyGov {
136         _lock(payee, lockAmount);
137     }
138
139     function lockMore(uint256 lockAmount) external onlyGovStaker {
140         _lock(msg.sender, lockAmount);
141
142         if(ncpStaking != address(0)){
143
          INCPStaking(ncpStaking).ncpDeposit(lockAmount, payable(msg.sender));
144         }
145     }
```

Scenario

If the locked amount exceeds the combined value of the minimum staking amount and the withdrawn amount, and the sender decides to invoke the `withdraw()` function, it will trigger the execution of the `ncpWithdraw()` function. However, if the sender has not previously invoked the `ncpDeposit()` function, this sequence of actions may result in a failed withdrawal for the sender.

Recommendation

We recommend adding the corresponding logic to invoke the `ncpDeposit()` function.

Alleviation

[Wemix Team]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

SIU-06 THE `unlock()` FUNCTION IS MISSING THE `ncpWithdraw()` FUNCTION CALL

Category	Severity	Location	Status
Logical Issue	Medium	StakingImp.sol: 183	Acknowledged

Description

In the `unlock()` function, both `_lockedBalance` and `_totalLockedBalance` are updated accordingly. In a scenario where a staker deposits ETH and triggers the `ncpDeposit()` function, subsequently, if the governance calls the `unlock()` function to release the staker's locked amount, and the staker subsequently calls the `withdraw()` function, it is important to note that the `ncpWithdraw()` function will not be invoked if the locked amount does not exceed the combined value of the minimum staking amount and the withdrawn amount.

Recommendation

We recommend adding the corresponding logic to invoke the `ncpWithdraw()` function.

Alleviation

[Wemix Team]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

CON-04 UPGRADEABLE CONTRACT EXTENDS NON-UPGRADEABLE CONTRACT

Category	Severity	Location	Status
Logical Issue	Minor	GovImp.sol: 13; StakingImp.sol: 8	Acknowledged

Description

Upgradeable contract must extend upgradeable contracts and call initializer functions of parent-contracts in its own initializer function. Otherwise it may malfunction due to improper initialization or improper upgrading.

Recommendation

We recommend using the `@openzeppelin/contracts-upgradeable/` instead of `@openzeppelin/contracts/`.

Alleviation

[Wemix Team]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

CON-06 NOT ALL PARENT CONTRACT INITIALIZING FUNCTIONS ARE CALLED

Category	Severity	Location	Status
Volatile Code	Minor	GovImp.sol: 20; StakingImp.sol: 14	Acknowledged

Description

Contract GovImp/StakingImp extends UUPSUpgradeable, while `__UUPSUpgradeable_init()` is not called in the initialize function. Generally, the initializer function of an upgradeable contract should always call all the initializer functions of the contracts that it extends.

Recommendation

We recommend calling `__UUPSUpgradeable_init()` in the linked contract.

Alleviation

[Wemix Team]: The `__UUPSUpgradeable_init()` function is missing in the current version, we will add it in a future update.

GIU-03 INTEGER UNDERFLOW WILL CAUSE `reInitV3()` TO REVERT

Category	Severity	Location	Status
Logical Issue	Minor	GovImp.sol: 1245	Resolved

Description

If `i` is equal to `0`, the code `address newReward = newRewards[i-1];` will result in an underflow and inadvertently cause the `reInitV3()` function to revert.

```
 1241      function reInitV3(uint256[] memory indices, address[] memory newRewards)
external reinitializer(3) onlyOwner{
 1242          unchecked {
 1243              for(uint256 i=0; i<indices.length;i++){
 1244                  address oldReward = rewards[i];
 1245                  address newReward = newRewards[i-1];
 1246                  rewards[i] = newReward;
 1247                  rewardIdx[newReward] = i;
 1248                  rewardIdx[oldReward] = 0;
 1249              }
 1250          }
 1251          //for the testnet
 1252          checkNodeName[nodes[40].name] = true;
 1253          checkNodeEnode[nodes[40].enode] = true;
 1254          checkNodeIpPort[keccak256(abi.encodePacked(nodes[40].ip, nodes[40].
port))] = true;
 1255      }
```

Recommendation

We recommend adjusting to ensure that the code handles the case when i is 0 without encountering underflow errors.

Alleviation

The client revised the code and resolved this issue in commit : 16849896b3bec89342823fb1aa39845d78996d12

GIU-04 | INCONSISTENT `lockAmount` CHECK

Category	Severity	Location	Status
Inconsistency	Minor	GovImp.sol: 111, 165	Resolved

Description

In the `initOnce()` function, the `lockAmount` is expected to fall within the range of the minimum and maximum staking amounts. However, in the `init()` function, the check only verifies that the `lockAmount` is greater than 0, without considering the specific staking range.

```
103     function init(
104         address registry,
105         uint256 lockAmount,
106         bytes memory name,
107         bytes memory enode,
108         bytes memory ip,
109         uint port
110      ) public initializer {
111         require(lockAmount > 0, "lockAmount should be more then zero");
112         ...
113      }
```

```
154  function initOnce(address registry, uint256 lockAmount, bytes memory data)
public initializer {
155         ...
156         require(lockAmount >= getMinStaking() && getMaxStaking() >= lockAmount,
 "Invalid lock amount");
157         ...
158  }
```

Recommendation

We recommend keeping the same check logic.

Alleviation

The client revised the code and resolved this issue in commit : 5a527d04e1cf5319649079b440aa0dea1dea2024

GIU-05 | LACK OF MODIFIER `checkTimePeriod`

Category	Severity	Location	Status
Volatile Code	Minor	GovImp.sol: 401	Resolved

Description

The modifier `checkTimePeriod` restricts the time between proposals to be greater than the specified proposal time period. This modifier is applied to the functions `addProposalToAddMember()`, `addProposalToRemoveMember()`, `addProposalToChangeMember()`, and `addProposalToChangeEnv()`. However, it is worth noting that the `addProposalToChangeGov()` function does not currently have this modifier applied.

Recommendation

We recommend including the `checkTimePeriod` modifier in the `addProposalToChangeGov()` function. This ensures consistency across all proposal-related functions and enforces the time period restriction as intended.

Alleviation

The client revised the code and resolved this issue in commit : fbdf6ceb5eeffa4d1300dcffa0bae79cefd6bd3b

GIU-06 VOTING THRESHOLD NOT DEPENDENT ON VOTING PARTICIPATION

Category	Severity	Location	Status
Logical Issue	Minor	GovImp.sol: 504	Acknowledged

Description

The threshold to pass or reject a vote is a hardcoded number 5001. If voting participation is low, then it is possible that neither `accept` nor `reject` reach the threshold in order to finalize the vote, and the ballot gets rejected when timestamp exceeds the voting `endTime`. If some stakers/voters are unable to vote either temporarily or permanently, then it could make it harder to pass a ballot than what is intended.

Recommendation

We recommend confirming if this is the intended / acceptable behavior. Consider including voting participation rate as a factor in determining whether a ballot passes or fails.

Alleviation

[Wemix Team]: According to our policy, this governance always requires a majority vote, otherwise the chain has a problem so the vote should not be accepted.

GIU-07 | SOME VARIABLES OF MEMBERINFO ARE NOT CHECKED

Category	Severity	Location	Status
Volatile Code	Minor	GovImp.sol: 85	Resolved

Description

The `checkMemberInfo` modifier validates the member info, however, it does not cover all the variables that need to be checked. Notably, the variables `enode` , `meno` , and `duration` are not included in the validation process.

Recommendation

We recommend checking the all the variables to avoid unexpected errors.

Alleviation

The client revised the code and resolved this issue in commit : e2deff1c07eb36c23c082d36bbce7ba3f11d530b

GIU-12 | OUT OF SCOPE DEPENDENCY

Category	Severity	Location	Status
Logical Issue	Minor	GovImp.sol: 9, 10	Resolved

Description

The `GovImp` contract serves as the underlying entity to interact with the external dependency `IBallotStorage` and `IEnvStorage`. The `BallotStorage` can create a ballot for address/variable/member, create a vote, update the ballot member lock amount, update the ballot memo, start the ballot, and finalize the ballot. The `EnvStorageImp` can check the variable condition and set the variable.

The scope of the audit treats component that is out of scope as black boxes and assumes their functional correctness.

Recommendation

The `BallotStorage` and `EnvStorageImp` are out of this audit scope. We encourage the team to constantly monitor the status of the those dependencies and ensure its security and functionality correctness.

Alleviation

[Wemix Team]: Excludes scopes that have been previously audited and have not been modified.

SIU-07 | USAGE OF `transfer` / `send` FOR SENDING ETHER

Category	Severity	Location	Status
Volatile Code	Minor	StakingImp.sol: 124, 247, 346	Partially Resolved

Description

It is not recommended to use Solidity's `transfer()` and `send()` functions for transferring Ether, since some contracts may not be able to receive the funds. Those functions forward only a fixed amount of gas (2300 specifically) and the receiving contracts may run out of gas before finishing the transfer. Also, EVM instructions' gas costs may increase in the future. Thus, some contracts that can receive now may stop working in the future due to the gas limitation.

Recommendation

We recommend using the `Address.sendValue()` function from OpenZeppelin.

Since `Address.sendValue()` may allow reentrancy, we also recommend guarding against reentrancy attacks by utilizing the Checks-Effects-Interactions Pattern or applying OpenZeppelin ReentrancyGuard.

Alleviation

The client revised the code and partially resolved this issue in commit : ea798f17b26b9577b35e883e8af706eb7d192967

SIU-08 | THIRD-PARTY DEPENDENCY USAGE

Category	Severity	Location	Status
Volatile Code	Minor	StakingImp.sol: 278	Resolved

Description

The contract is serving as the underlying entity to interact with one or more third party protocols. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

```
278     address public ncpStaking;
```

- The contract `StakingImp` interacts with third party contract with `INCPStaking` interface via `ncpStaking`.

Recommendation

The auditors understood that the business logic requires interaction with third parties. It is recommended for the team to constantly monitor the statuses of third parties to mitigate the side effects when unexpected activities are observed.

Alleviation

[Wemix Team]: This ncpStaking was audited by the wonder staking project.

SIU-09 MISSING CHECK `unlockAmount` IS LESS THAN `_lockedBalance`

Category	Severity	Location	Status
Volatile Code	Minor	StakingImp.sol: 183, 187	Resolved

Description

Currently, if the `unlockAmount` exceeds the `_lockedBalance`, the `_lock()` function will revert without providing a user-friendly error message. Furthermore, it is worth noting that when the governance invokes the `unlock()` function, there is no minimum staking amount constraint.

Recommendation

We recommend adding a check in the `_lock()` function to ensure that the `unlockAmount` is not greater than the `_lockedBalance`.

Alleviation

The client revised the code and resolved this issue in commit : 8fc620cad664aee6e0f0ba3a41351c0001f5ad50

CON-05 | MISSING EMIT EVENTS

Category	Severity	Location	Status
Coding Style	Informational	GovImp.sol: 1230, 1241; StakingImp.sol: 300	Resolved

Description

There should always be events emitted in the sensitive functions that are controlled by centralization roles.

Recommendation

It is recommended emitting events for the sensitive functions that are controlled by centralization roles.

Alleviation

The client revised the code and resolved this issue in commit : 07edff67922c1fe76dbf5ce42c8fc072cfdcf684

GIU-08 | REDUNDANT CODE

Category	Severity	Location	Status
Coding Style	Informational	GovImp.sol: 703	Resolved

Description

The update of the node's name has already been completed in line 692. However, line 703 performs the same operation redundantly, which is unnecessary and can be removed to avoid duplication.

Recommendation

We recommend removing the redundant code.

Alleviation

The client revised the code and resolved this issue in commit : d07ae4d469c4d7fea1cfb873bb559dddf64746a3

GIU-09 | LACK VALIDATION FOR ARRAY LENGTH

Category	Severity	Location	Status
Logical Issue	Informational	GovImp.sol: 1241	Resolved

Description

There is no validation to check whether `indices.length` , and `newRewards.length` are the same.

Recommendation

We recommended adding validation like the below:

```
require(indices.length == newRewards.length,"indices and newRewards len mismatch");
```

Alleviation

The client revised the code and resolved this issue in commit : 16849896b3bec89342823fb1aa39845d78996d12

GIU-10 | CODE FOR TEST NET

Category	Severity	Location	Status
Logical Issue	Informational	GovImp.sol: 1251~1254	Resolved

Description

The comment "//for the testnet" serves as a clear indication that the following code is exclusively intended for the test network. It is crucial to remove these specific lines of code before releasing the main network to ensure that they are not accidentally used, thus preventing any potential issues or unintended consequences on the live network.

```
 1241      function reInitV3(uint256[] memory indices, address[] memory newRewards)
external reinitializer(3) onlyOwner{
 1242          ...
 1243          //for the testnet
 1244          checkNodeName[nodes[40].name] = true;
 1245          checkNodeEnode[nodes[40].enode] = true;
 1246          checkNodeIpPort[keccak256(abi.encodePacked(nodes[40].ip, nodes[40].
port))] = true;
 1247      }
```

Recommendation

We recommend removing these lines of code entirely.

Alleviation

The client revised the code and resolved this issue in commit : 16849896b3bec89342823fb1aa39845d78996d12

GIU-11 | DISCUSSION ON `weight`

Category	Severity	Location	Status
Logical Issue	Informational	GovImp.sol: 575	Resolved

Description

The calculation of `weight` involves dividing 10000 by the number of members. Consequently, the value of `weight` remains consistent as long as the number of members remains unchanged.

```
 575  uint256 weight = 10000 / getMemberLength();
//IStaking(getStakingAddress()).calcVotingWeightWithScaleFactor(staker, 10000);
```

Recommendation

We would like to confirm with the client if the current implementation aligns with the original project design.

Alleviation

[Wemix Team]: In phase2, we're keeping the number of governance members fixed at 40, so we set it up like this.

SIU-10 STATE VARIABLES IN UPGRADEABLE CONTRACTS ARE INITIALIZED WHEN DECLARED

Category	Severity	Location	Status
Logical Issue	Informational	StakingImp.sol: 20	Resolved

Description

State variables initialized when declared are equivalent to initializing them inside the constructor. Therefore, initializing state variables when declared in an upgradeable contract has no actual effect since the constructor of an upgradeable contract is never called.

Recommendation

We recommend initializing state variables in an initializer function if necessary to avoid unexpected behavior and confusion.

Alleviation

The client revised the code and resolved this issue in commit : fa5bd133197a8af0567c6b464b6df7148ec9da08

APPENDIX | WEMIX GOVERNANCE-CONTRACT RETAINER 35

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

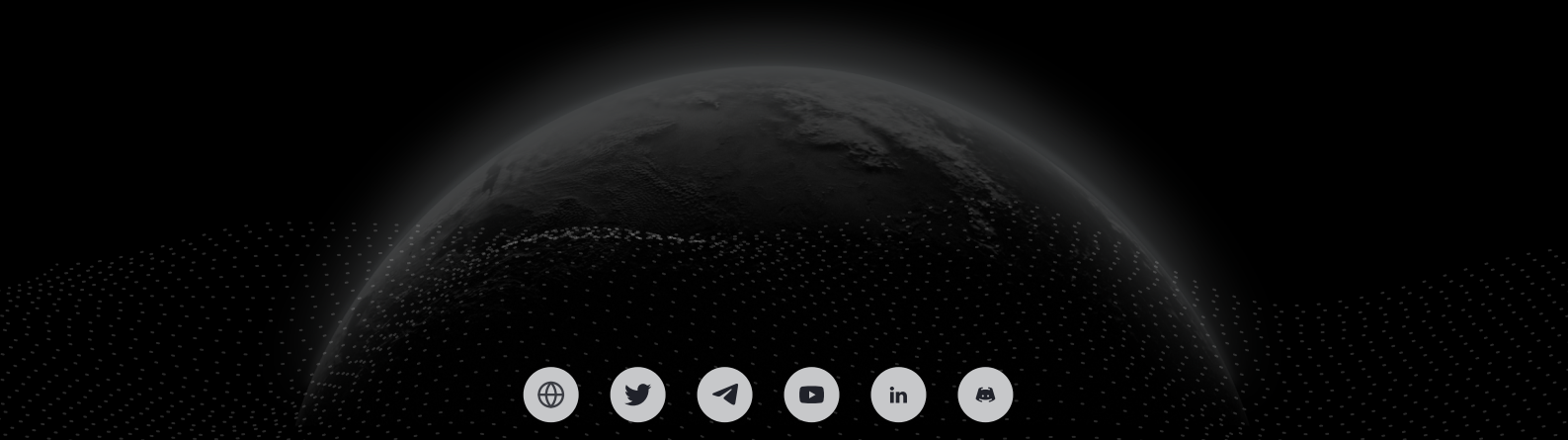
NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.



Copyright © CertiK